# *MONGO DB*

## 1. to show all available databases:

show dbs

show databases

```
> show dbs
< Companydb  220.00 KiB
  admin       40.00 KiB
  config      36.00 KiB
  local       64.00 KiB
```

---

## 2. to display database you are using:

db

note: it should return test - the default data base -

```
> db
< test
```

---

## 3. switch database:

use <db>

ex: use students

```
> use students
< switched to db students
students>
```

---

4. clear the mongosh console:

cls

ctrl + l

---

<u>CRUD operations:</u>

create || Insert operation:

5. to insert a single document:

db.collection.insertOne()

```
< Companydb  220.00 KiB
  admin       40.00 KiB
  config      96.00 KiB
  local       64.00 KiB
> db.section1.insertOne({
    name : "asmaa",
    age  : 21,
    major: "IS"
  })
< {
    acknowledged: true,
    insertedId: ObjectId('69efb838753ca592053066d6')
  }
```

```
> show databases
< Companydb  220.00 KiB
  admin       40.00 KiB
  config      96.00 KiB
  local       64.00 KiB
  students     8.00 KiB
students>
```

---

## 6. to insert multiple documents:

db.collection.insertMany( [ {.....} ] )

ex:

```
> db.section1.insertMany([
    {
      name: "mirit",
      age: 21,
      major: "IS"
    },
    {
      name : 'layla',
      age  :  20,
      major:  'AI'
    },
    {
      name : 'angham',
      age  :   20,
      major:   'arts'
    }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('69efb986753ca592053066d7'),
      '1': ObjectId('69efb986753ca592053066d8'),
      '2': ObjectId('69efb986753ca592053066d9')
    }
  }
```

---

## Read:

7. to read all documents in the collection:

db.collection.find()

ex:

```
>_MONGOSH
> db.section1.find()
< {
    _id: ObjectId('69efb838753ca592053066d6'),
    name: 'asmaa',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d7'),
    name: 'mirit',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d8'),
    name: 'layla',
    age: 20,
    major: 'AI'
  }
  {
    _id: ObjectId('69efb986753ca592053066d9'),
    name: 'angham',
    age: 20,
    major: 'arts'
  }
```

---

## 8. to select documents which match an equality condition

db.collection.find( { field : value } )

ex: db.section1.find( { name : 'mirit' } )

```
> db.section1.find({ name: "mirit" })
< {
    _id: ObjectId('69efb986753ca592053066d7'),
    name: 'mirit',
    age: 21,
    major: 'IS'
  }
```

---

## 9. limit the collection to just (2)

db.collection.find().limit( number of documents)

ex: db.section1.find().limit(2)

```
> db.section1.find().limit(2)
< {
    _id: ObjectId('69efb838753ca592053066d6'),
    name: 'asmaa',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d7'),
    name: 'mirit',
    age: 21,
    major: 'IS'
  }
```

---

## 10. skip collection:

db.collection.find().skip( number of the documents )

ex: db.section1.find().skip(2)

```
> db.section1.find().skip(2)
< {
    _id: ObjectId('69efb986753ca592053066d8'),
    name: 'layla',
    age: 20,
    major: 'AI'
  }
  {
    _id: ObjectId('69efb986753ca592053066d9'),
    name: 'angham',
    age: 20,
    major: 'arts'
  }
```

---

## Query Operators

## 1.Comparison Operators

- `$eq` : Values are equal
- `$ne` : Values are not equal
- `$gt` : Value is greater than another value
- `$gte` : Value is greater than or equal to another value
- `$lt` : Value is less than another value
- `$lte` : Value is less than or equal to another value
- `$in` : Value is matched within an array

db.collection.find( { lhs : { $operator : rhs} } )

note: rhs => right hand side  == attribute

lhs => left hand side  == value

## 11. equal:

db.collection.find( { attribute: { $eq : value} } )

db.collection.find( { age : {$eq : 20} } )

```
> db.section1.find({ age : {$eq : 20} })
< {
    _id: ObjectId('69efb986753ca592053066d8'),
    name: 'layla',
    age: 20,
    major: 'AI'
  }
  {
    _id: ObjectId('69efb986753ca592053066d9'),
    name: 'angham',
    age: 20,
    major: 'arts'
  }
```

## 12. not equal:

db.collection.find( {attribute : {$ne :  value} } )

db.section1.find( { age : {$ne : 20 } } )

```
> db.section1.find( {age : { $ne : 20 }} )
< {
    _id: ObjectId('69efb838753ca592053066d6'),
    name: 'asmaa',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d7'),
    name: 'mirit',
    age: 21,
    major: 'IS'
  }
```

## 13. greater than:

db.collection.find( {attribute : {$gt : value} } )

db.section1.find( { age : {$gt : 19} } )

```
> db.section1.find({age : {$gt : 20}})
< {
    _id: ObjectId('69efb838753ca592053066d6'),
    name: 'asmaa',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d7'),
    name: 'mirit',
    age: 21,
    major: 'IS'
  }
```

## 14. greater than or equal:

db.collection.find( {attribute : {$gte : value} } )

db.section1.find( { age : {$gte : 19} } )

```
> db.section1.find({age : {$gte : 19}})
< {
    _id: ObjectId('69efb838753ca592053066d6'),
    name: 'asmaa',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d7'),
    name: 'mirit',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d8'),
    name: 'layla',
    age: 20,
    major: 'AI'
  }
  {
    _id: ObjectId('69efb986753ca592053066d9'),
    name: 'angham',
    age: 20,
    major: 'arts'
  }
```

## 15. less than:

db.collection.find( {attribute : {$lt : value} } )

db.section1.find( { age : {$lt : 21} } )

```
> db.section1.find({age : {$lt : 21}})
< {
    _id: ObjectId('69efb986753ca592053066d8'),
    name: 'layla',
    age: 20,
    major: 'AI'
  }
  {
    _id: ObjectId('69efb986753ca592053066d9'),
    name: 'angham',
    age: 20,
    major: 'arts'
  }
```

## 16. less than or equal:

db.collection.find( {attribute : {$lte : value} } )

db.section1.find( { age : {$lte : 21} } )

```
> db.section1.find({age : {$lte : 21}})
< {
    _id: ObjectId('69efb838753ca592053066d6'),
    name: 'asmaa',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d7'),
    name: 'mirit',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d8'),
    name: 'layla',
    age: 20,
    major: 'AI'
  }
  {
    _id: ObjectId('69efb986753ca592053066d9'),
    name: 'angham',
    age: 20,
    major: 'arts'
  }
```

## 17. in:

db.collection.find( {attribute : {$in : value} } )

db.section1.find( { age : {$in :  [20, 21, 22] } } )

```
> db.section1.find({age : {$in : [20,21]}})
< {
    _id: ObjectId('69efb838753ca592053066d6'),
    name: 'asmaa',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d7'),
    name: 'mirit',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d8'),
    name: 'layla',
    age: 20,
    major: 'AI'
  }
  {
    _id: ObjectId('69efb986753ca592053066d9'),
    name: 'angham',
    age: 20,
    major: 'arts'
  }
```

---

## Logical Operators :

- $and : Returns documents where both queries match
- $or : Returns documents where either query matches
- $nor : Returns documents where both queries fail to match
- $not : Returns documents where the query does not match

db.collection.find( { $operator :[ { condition1 } , { condition2  } .... ] } )

## 18. and:

ex:

```
> db.section1.find(
    {
      $and : [
        {age : {$gt : 20}},
        {major : "IS"}
      ]
    }
  )
< {
    _id: ObjectId('69efb838753ca592053066d6'),
    name: 'asmaa',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d7'),
    name: 'mirit',
    age: 21,
    major: 'IS'
  }
```

## 19. or:

db.collection.find({ $or : [ {condition1} ],
{condition2} ] } )

```
> db.section1.find({
    $or : [
      {age : {$gt : 20}},
      {major : "IS"}
    ]
  })
< {
    _id: ObjectId('69efb838753ca592053066d6'),
    name: 'asmaa',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d7'),
    name: 'mirit',
    age: 21,
    major: 'IS'
  }
```

---

## 20. nor

```
db.collection.find({
$nor: [
{ condition1 },
 { condition2 }
        ]
})
```

```
> db.section1.find({
    $nor : [
      {name : "asmaa"},
      {age  : {gt : 20}}
    ]
  })
< {
    _id: ObjectId('69efb986753ca592053066d7'),
    name: 'mirit',
    age: 21,
    major: 'IS'
  }
  {
    _id: ObjectId('69efb986753ca592053066d8'),
    name: 'layla',
    age: 20,
    major: 'AI'
  }
  {
    _id: ObjectId('69efb986753ca592053066d9'),
    name: 'angham',
    age: 20,
    major: 'arts'
  }
```

---

## 20. not:

db.collection.find({attribute : {$not : {condition} } } )

ex:

```
> db.section1.find({
    age : {$not : {$eq : 21}}
  })
< {
    _id: ObjectId('69efb986753ca592053066d8'),
    name: 'layla',
    age: 20,
    major: 'AI'
  }
  {
    _id: ObjectId('69efb986753ca592053066d9'),
    name: 'angham',
    age: 20,
    major: 'arts'
  }
```

---

## Update:

note: increase operators:

set , inc , mul

21. to update a sigle document:

db.collection.updateOne( { condition } ,

$update_operator : { update } )

ex: db.section1.updateOne( {name : 'angham'},

{ $set : { age : 21 } })

```
db.section1.updateOne(
  {name : "asmaa"},
  {$set : {age : 22}}
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
_id: ObjectId('69efb838753ca592053066d6')
name : "asmaa"
age : 22
major : "IS"
```

---

## 22. update multiple documents:

db.collection.updateMany( { condition == filter} ,

$set : { update } )

ex: db.section1.updateMany({age : {$gt : 20}},

{$set : {major : 'AI'}})

db.section1.updateMany( { age : {$gt : 20} },

{$inc : {age : 1}})

note: update all documents without condition:

db.collection.updateOne( { } , $set : { update } )

```
> db.section1.updateMany(
    {age : {$gt : 20}},
    {$set : {salary : 20000}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 2,
    modifiedCount: 2,
    upsertedCount: 0
  }
```

```
_id: ObjectId('69efb838753ca592053066d6')
name : "asmaa"
age : 22
major : "IS"
salary : 20000

_id: ObjectId('69efb986753ca592053066d7')
name : "mirit"
age : 21
major : "IS"
salary : 20000

_id: ObjectId('69efb986753ca592053066d8')
name : "layla"
age : 20
major : "AI"

_id: ObjectId('69efb986753ca592053066d9')
name : "angham"
age : 20
major : "arts"
```

---

## 23. replace:

db.collection.replaceOne( {condition} , {new

document} )

```
> db.section1.replaceOne(
    {age : {$eq : 22}},
    {
      name : "angham",
      major : "CS"
    }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

```
_id: ObjectId('69efb838753ca592053066d6')
name : "angham"
major : "CS"
```

---

## Delete Operation:

24. to delete a single document:

db.collection.deleteOne( {condition } )

ex: db.section1.deleteOne( {name : 'asmaa'} )

note:

⚠ Important notes for the quiz:

deleteOne() deletes only the first document that matches

the condition.

If multiple documents match the same condition, only one document will be deleted.

If no document matches the condition, nothing will be deleted.

```
> db.section1.deleteOne({name : "asmaa"})
< {
    acknowledged: true,
    deletedCount: 0
  }
```

---

25. to delete multiple documents:

db.collection.deleteMany( {condition } )

ex: db.section1.deleteMany( {name : 'mirit'} )

```
> db.section1.deleteMany(
    {age : {$gt : 19}}
  )
< {
    acknowledged: true,
    deletedCount: 2
  }
```

---

26. drop a collection:

db.collection.drop()

ex: db.students1.drop()

```
> db.section1.drop()
< true
```

---

27. drop a database:

use database_name

db.dropDatabase()

ex: use students

db.dropDatabase()

```
> db.dropDatabase()
< { ok: 1, dropped: 'students' }
```

---
--

---
---
---

**Aggregate piplines:**

28. match:

db.collection.aggregate([

{ $match: { condition } }

])

ex:

```
> db.students.aggregate([
    {$match : {salary : {$gt : 1000}}}
  ])
< {
    _id: ObjectId('69efcd1e753ca592053066da'),
    name: 'asmaa',
    age: 21,
    major: 'IS',
    salary: 1200
  }
  {
    _id: ObjectId('69efcd1e753ca592053066dc'),
    name: 'layla',
    age: 22,
    major: 'CS',
    salary: 1500
  }
  {
    _id: ObjectId('69efcd1e753ca592053066de'),
    name: 'youssef',
    age: 23,
    major: 'IS',
    salary: 2000
  }
```

---

## 29. group:

db.collection.aggregate([
{ $group : { _id : {$fild} },

new_attribute : { $operator: "$field"}  } ])

ex:

```
> db.students.aggregate([
    {
      $group: {
        _id : "$major",
        avesalary : {$avg : "$salary"}
      }
    }
  ])
< {
    _id: 'AI',
    avesalary: 900
  }
  {
    _id: 'IS',
    avesalary: 1600
  }
  {
    _id: 'CS',
    avesalary: 1500
  }
  {
    _id: 'Arts',
    avesalary: 700
  }
```

---

## 30. project:

db.collection.aggregate([

```
  {
    $project: {
      field1: 1,
      field2: 0
    }
  }
])
```

note: 0 => disappeare // exclusion

1 => appeare // inclusion

id appears by default

ex:

```
> db.students.aggregate([
    {
      $project: {salary : 0 , major : 0}
    }
  ])
< {
    _id: ObjectId('69efcd1e753ca592053066da'),
    name: 'asmaa',
    age: 21
  }
  {
    _id: ObjectId('69efcd1e753ca592053066db'),
    name: 'mirit',
    age: 20
  }
  {
    _id: ObjectId('69efcd1e753ca592053066dc'),
    name: 'layla',
    age: 22
  }
  {
    _id: ObjectId('69efcd1e753ca592053066dd'),
    name: 'angham',
    age: 19
  }
  {
    _id: ObjectId('69efcd1e753ca592053066de'),
    name: 'youssef',
    age: 23
```

---

## 31. sort:

```
db.collection.aggregate([
  {
    $sort: { attribute: -1 }
```

}

])

note: 1 => Ascending

-1 => Descending

ex:

```
> db.students.aggregate([
    {
      $sort : {salary : -1},
    },
    {$skip : 1},
    {$limit: 2}
  ])
< {
    _id: ObjectId('69efcd1e753ca592053066dc'),
    name: 'layla',
    age: 22,
    major: 'CS',
    salary: 1500
  }
  {
    _id: ObjectId('69efcd1e753ca592053066da'),
    name: 'asmaa',
    age: 21,
    major: 'IS',
    salary: 1200
  }
```

---

## 32. unwind:

db.collection.aggregate([

{ $unwind: "$field" }

])

ex:

```
> db.students.updateMany(
    {},
    {$set : {hobbies : ['dance' , 'reading' , 'sport']}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 5,
    modifiedCount: 5,
    upsertedCount: 0
  }
> db.students.aggregate(
    [
      {$unwind : "$hobbies"}
    ]
  )
```

```
_id: ObjectId('69efcd1e753ca592053066da')
name : "asmaa"
age : 21
major : "IS"
salary : 1200
▸ hobbies : Array (3)

_id: ObjectId('69efcd1e753ca592053066db')
name : "mirit"
age : 20
major : "AI"
salary : 900
▸ hobbies : Array (3)

_id: ObjectId('69efcd1e753ca592053066dc')
name : "layla"
age : 22
major : "CS"
salary : 1500
▸ hobbies : Array (3)
```

---

## -33. addfields & set:

db.collection.aggregate([
{$addFields : {new_attribute : {arthmatic operation}}} ])

ex:

```
> db.students.aggregate([
    {$addFields : {tax : {$multiply : ["$salary" , 0.1]}}}
  ])
< {
    _id: ObjectId('69efcd1e753ca592053066da'),
    name: 'asmaa',
    age: 21,
    major: 'IS',
    salary: 1200,
    hobbies: [
      'dance',
      'reading',
      'sport'
    ],
    tax: 120
  }
  {
    _id: ObjectId('69efcd1e753ca592053066db'),
```

```
> db.students.aggregate([
    {$set : {tax : {$multiply : ["$salary" , 0.1]}}}
  ])
< {
    _id: ObjectId('69efcd1e753ca592053066da'),
    name: 'asmaa',
    age: 21,
    major: 'IS',
    salary: 1200,
    hobbies: [
      'dance',
      'reading',
      'sport'
    ],
    tax: 120
  }
```

---

## 34. lookup:

db.collection1.aggregate([
{$lookup : {

from : collection2,

localField : "primary key name",

foreignField: "foriegn key name",

as : "attribute name" } }])

ex:

```
> db.students.aggregate([
    {
      $lookup: {
        from: "students2",
        localField: "salary",
        foreignField: "salary",
        as: "salary_info"
      }
    }
  ])
< {
    _id: ObjectId('69efcd1e753ca592053066da'),
    name: 'asmaa',
    age: 21,
    major: 'IS',
    salary: 1200,
    hobbies: [
      'dance',
      'reading',
      'sport'
    ],
    salary_info: [
      {
        _id: ObjectId('69efd760753ca592053066e0'),
        name: 'Jilan',
        grad: 'A+',
        salary: 1200
      }
    ]
```

---

## -35. merge:

```
db.collection.aggregate([
  {
    $merge: {
```

```
      into: "target_collection",
      whenMatched: "merge",
      whenNotMatched: "insert"
    }
  }
])
```

ex:

```
> db.students.aggregate([
    {
      $merge: {
        into: "students_backup",
        whenMatched: "merge",
        whenNotMatched: "insert"
      }
    }
  ])
<
```

---

-36. sample:

```
db.employee.aggregate([
  { $sample: { size: number } }
])
```

ex:

```
> db.students.aggregate([
    { $sample: { size: 2 } }
  ])
< {
    _id: ObjectId('69efcd1e753ca592053066da'),
    name: 'asmaa',
    age: 21,
    major: 'IS',
    salary: 1200,
    hobbies: [
      'dance',
      'reading',
      'sport'
    ]
  }
  {
    _id: ObjectId('69efcd1e753ca592053066db'),
    name: 'mirit',
    age: 20,
    major: 'AI',
    salary: 900,
    hobbies: [
      'dance',
      'reading',
      'sport'
    ]
  }
```

---

## 37. count

db.collection.aggregate([{$count : "attribute name"}])

ex:

```
> db.students.aggregate([
    { $count: "total_count" }
  ])
< {
    total_count: 5
  }
students>
```

---

## -38. sort by count:

db.collection.aggregate([
{$sortByCount : "$attribute"} ])

ex:

```
> db.students.aggregate([
    { $sortByCount: "$major" }
  ])
< {
    _id: 'IS',
    count: 2
  }
  {
    _id: 'AI',
    count: 1
  }
  {
    _id: 'CS',
    count: 1
  }
  {
    _id: 'Arts',
    count: 1
  }
```

---

## 39. bucketAuto:

```
db.collection.aggregate([
{$bucketAuto : {groupBy : "$attribute name",
buckets : number} ])
```

ex:

```
> db.students.aggregate([
    {
      $bucketAuto: {
        groupBy: "$salary",
        buckets: 3
      }
    }
  ])
< {
    _id: {
      min: 700,
      max: 1200
    },
    count: 2
  }
  {
    _id: {
      min: 1200,
      max: 2000
    },
    count: 2
  }
  {
    _id: {
      min: 2000,
      max: 2000
    },
    count: 1
  }
```

---